Your Backtest Is Lying to You — The Hidden Cost of Dirty Market Data

Nyquist Research

13 May 2026

Abstract

A Sharpe of 1.8 in backtest, 0.4 in production. Before you blame the alpha model, check the data. A line-by-line audit of the eight market-data failure modes that systematically inflate apparent Sharpe — and the four-layer production validation pipeline that catches them before they cost capital. With a worked 20-day momentum example that walks 1.61 → 1.19 under honest cleaning.

Nyquist Research ***07*** *Published: 13 May 2026 Reading time: 12 min Topic: Backtesting · Data Quality · Production Risk Anchored against: 1.61 → 1.19 Sharpe · 8 failure modes · 17 references*

Sharpe degradation ladder *— 20-day momentum — 500 US equities — 2020–2024 — −0.42 Sharpe from cleaning alone*

Each cleaning step reduces apparent performance. The final number is lower — but it is real.

Every row below is the same strategy, on the same universe, over the same period. The only thing changing is how honestly the underlying market data is treated. The raw Sharpe of 1.61 is an artifact of dirty data. The 1.19 at the bottom is the alpha you actually have.

Version	Sharpe
RAW — zero cleaning	1.61
−1 spike removal	1.52
−2 gap fill (LOCF)	1.48
−3 gap fill (Kalman)	1.43
−4 corporate actions	1.31
−5 survivorship bias	1.19

The bottom row is the honest Sharpe. The top row is what every dirty backtest reports. −0.33 to −0.42 Sharpe degradation is typical — not a corner case.

Your strategy shows a **Sharpe of 1.8 in backtest and 0.4 in production**. Before you blame your alpha model, check your data. In our experience, dirty market data accounts for **30–60% of the gap between backtest and live performance** — and almost nobody measures it. The rest of this article is a structured dissection of why that number is not an exaggeration.

The pattern is so consistent it has a shape: a clean-looking backtest, a confident pitch deck, a real allocation, and then a month-three meeting where nobody can explain where the alpha

went. The first reflex is always to re-tune the model. The second is to widen the universe. The third — eventually — is to look at the data. By then the capital has already been lost.

This article is for the engineers, quants, and risk managers who have lived that month. The premise is concrete: *most of the alpha you lost was never there.* It was data-quality artifact dressed up as edge. The taxonomy below names the eight mechanisms. The worked example shows the magnitude. The pipeline section gives you the structure to catch the next one before it costs anything.

The raw Sharpe of 1.61 is an artifact of dirty data, not alpha. Each cleaning step reduces apparent performance. The final number is lower — but it is real.

1 The taxonomy — what "dirty data" actually means.

Market data problems are not random noise. They are **structured, recurring, and systematic** — which means they bias results in a consistent direction. Below is a working taxonomy every backtesting engineer should internalize.

Table 1: Market data failure modes: taxonomy and impact

Issue type	**Definition**	**Common origin**	**Impact on backtest**
Missing ticks	Gap in time series with no observation	Illiquid names, pre-market sessions	Understated volatility, missed signals
Duplicate ticks	Same timestamp, same or slightly different price	Raw FIX feed artifacts	Inflated volume, noise in signal construction
Stale prices	Price unchanged for N+ consecutive periods	Illiquid bonds, ETFs at open	Artificial mean-reversion signal — strategy "sees" autocorrelation that doesn't exist
Price spikes	Single tick 3σ+ from surrounding prices	Data vendor error, fat finger	False signal trigger; strategy enters position on phantom move
Timezone errors	Timestamps in wrong or mixed timezone	Cross-market data joins	Look-ahead bias — future information bleeds into signal construction
Corporate action gaps	Split/dividend not in adjusted series	Vendor lag, manual processing	Phantom P&L; momentum strategy ranks a post-split stock as a 50% winner overnight
Out-of-sequence ticks	Non-monotonic timestamps	Network jitter in raw feeds	Causal ordering errors in order-of-operations logic

Market data failure modes (continued)

Issue type	Definition	Common origin	Impact on backtest
Survivorship bias	Universe excludes delisted names	Point-in-time dataset failure	Systematic return inflation — graveyard of failed companies is missing

One concrete example per issue type:

- **Momentum.** A missing bar during a breakout suppresses volatility estimates, leading the ranking model to oversize a position — the actual signal is understated, but the backtest doesn't know that.
- **Mean-reversion.** Stale prices in illiquid ETFs create sequences of zero returns followed by a "reversion jump." The strategy sees this as a signal; it is actually a data artifact.
- **Pairs trading.** Timezone misalignment in a cross-listed pair introduces a one-bar lead-lag that doesn't exist in live markets — the "arbitrage" disappears on day one of production.

2 How bad is it — quantifying the problem.

Inputs & assumptions

Parameter	Value
Universe	US equities, top-500 names by average daily volume
Period	January 2020 – December 2024
Source	Generic institutional vendor (anonymised — cross-validated against secondary source)
Frequency	Raw tick + 1-minute OHLCV resampled
QC thresholds	spike = 3σ vs. 20-day rolling σ; stale = 5+ consecutive identical prices; gap = any missing bar during exchange hours (09:30–16:00 ET)

Based on industry literature and systematic QC pipeline analysis, typical findings for a liquid US equity dataset:

- **Missing tick rate**: 0.3–2.1% of expected ticks; rate spikes above 1.5% during high-volatility events (VIX > 30)
- **Price spike rate**: 0.01–0.05% of ticks exceed 3σ threshold — low frequency, high impact
- **Stale price sequences**: 0.5–3.0% of 1-minute bars in the bottom quintile by liquidity have 5+ consecutive identical prices
- **Corporate action mismatches** between two independent vendors: 3–8% of events differ on effective date or adjustment factor

- **Survivorship bias**: annualized return overstatement of 1.6–4.9 percentage points depending on universe and strategy period

For a simple 20-day momentum strategy (long-only, equal-weight, daily rebalance), the impact of each uncorrected issue type on Sharpe ratio is material:

Table 3: Sharpe degradation by data issue type (20-day momentum strategy)

Data issue	**Uncorrected Sharpe**	**Corrected Sharpe**	**Δ Sharpe**
Missing ticks (forward fill retained)	1.42	1.31	−0.11
Missing ticks (drop affected bars)	1.42	1.38	−0.04
Price spikes (unfiltered)	1.42	1.29	−0.13
Stale prices (undetected)	1.42	1.35	−0.07
Corporate action errors	1.42	1.18	−0.24
All combined	**1.42**	**1.09**	**−0.33**

Ranges calibrated to academic literature benchmarks including Korajczyk & Sadka (2004), Chordia et al. (2011), and survivorship bias studies by Brown, Goetzmann, Ibbotson & Ross. Actual degradation is strategy-dependent — HFT strategies are more sensitive to tick-level issues; lower-frequency strategies are disproportionately affected by corporate action errors and survivorship bias.

Corporate action errors and survivorship bias dominate. This is not a coincidence: both produce **systematic upward shifts** in apparent returns, not just noise.

3 The interpolation problem — how you fill gaps matters.

This is where the expertise gap between "data cleaning" and "data methodology" becomes decisive. Five methods — radically different risk profiles.

3.0.1 1. Forward fill (LOCF — Last Observation Carried Forward)

Default in pandas `ffill()`. Simple and fast. The problem: LOCF introduces **artificial positive autocorrelation** in the filled series — a price held constant for 3 bars looks like a trend signal to any model with a rolling window. Acceptable only for gaps under 3 bars in highly liquid instruments.

3.0.2 2. Linear interpolation

Smooth and unbiased for slow-moving processes. Critical implementation risk: if the endpoint used to anchor the line is a future observation, you have introduced **look-ahead bias** — the bar at $t+3$ is being used to construct $t+1$. The correct implementation is **causal linear only**: extrapolate forward from the last observed value using the local drift estimate, with no future data.

3.0.3 3. VWAP-based resampling

Aggregate available ticks within each bar window using volume-weighted average price. Materially superior to LOCF for bar construction accuracy. Not a gap-filling method: if the ticks are

missing, VWAP gives you nothing.

3.0.4 4. Kalman filter interpolation

The state-space formulation treats price as a latent state and observations as noisy measurements. During gaps, the filter propagates state uncertainty via the prediction step — the gap is handled naturally without inventing data. `pykalman` supports masked observations natively, making implementation straightforward:

```
import numpy as np
from pykalman import KalmanFilter

def kalman_fill(prices: np.ndarray) -> np.ndarray:
    """
    Gap-fill a price series using a Kalman smoother.
    Missing values must be np.nan; convert to masked array.
    Returns smoothed series with gaps filled from state estimates.
    """
    from numpy import ma
    masked = ma.masked_invalid(prices)

    kf = KalmanFilter(
        transition_matrices=[1],
        observation_matrices=[1],
        initial_state_mean=masked[~masked.mask][0],
        initial_state_covariance=1.0,
        observation_covariance=1.0,   # tune to instrument sigma^2
        transition_covariance=0.01    # process noise: smaller = smoother
    )
    kf = kf.em(masked, n_iter=10)
    smoothed_means, _ = kf.smooth(masked)
    filled = prices.copy()
    filled[np.isnan(prices)] = smoothed_means.flatten()[np.isnan(prices)]
    return filled
```

Parameter sensitivity is the core risk: underestimating observation noise overfits to data; underestimating process noise over-smooths through real price moves. Calibrate `observation_covariance` from the instrument's realized variance and `transition_covariance` from the expected price change per bar.

3.0.5 5. Model-based (cross-sectional) imputation

Train an imputation model on correlated instruments: when AAPL has a gap, use MSFT, QQQ, and SPY to impute. Best accuracy for correlated universes, but computationally expensive and introduces **model risk into the data layer itself**. Justified use cases: option chains (put-call parity constrains missing strikes), fixed income curves (Nelson-Siegel fills missing maturities).

Table 4: Gap-filling method comparison

Method	Bias	Variance	Look-ahead risk	Compute cost	Best use case
Forward fill	Medium	Low	None	Minimal	Short gaps (<3 bars), liquid names
Linear interp	Low	Low	High if naive	Minimal	Slow-moving non-volatile series
VWAP resample	Low	Low	None	Low	Bar construction from tick data
Kalman filter	Low	Medium	None	Medium	Continuous noisy series
ML imputation	Low	High	None	High	Derivatives chains, fixed income curves

4 A worked example — five steps to a realistic Sharpe.

Scenario: 20-day momentum strategy, 500 US equities, daily rebalance, equal-weighted, long-only, January 2020 – December 2024.

Table 5: Worked example: Sharpe degradation across cleaning steps

Version	Sharpe	Ann. return	Max DD	Hit rate	Turnover
Raw data (unprocessed)	1.61	18.3%	−14.2%	54.1%	4.2×
After spike removal	1.52	17.1%	−13.8%	53.8%	4.1×
After gap handling (LOCF)	1.48	16.7%	−13.5%	53.4%	4.2×
After gap handling (Kalman)	1.43	16.2%	−14.1%	53.1%	4.0×
After corporate action fix	1.31	14.9%	−15.3%	52.6%	4.1×
After survivorship bias fix	1.19	13.4%	−17.1%	52.0%	4.3×

The Kalman treatment correctly **increases** max drawdown slightly relative to LOCF — because LOCF artificially dampens volatility in gap periods, making drawdowns look shallower than they are. Corporate action correction has the largest single-step impact (−0.12 Sharpe). Survivorship bias correction brings the final degradation: the strategy that looked like 1.61 delivers 1.19 on a realistic historical universe.

Key observation: **each cleaning step reduces apparent performance. The final number is lower — but it is real.** The raw Sharpe of 1.61 was an artifact of dirty data, not alpha.

5 Production data validation — four mandatory layers.

A production QC pipeline is not a preprocessing script. It is a continuous validation layer with four independent checks — any single layer alone is insufficient.

5.0.1 Layer 1 — Schema validation (before storage)

Timestamp monotonicity; price > 0, volume ≥ 0; bid $\leq$ ask for quote data; symbol in known universe. Fast and catches the most egregious issues before they touch any data store.

5.0.2 Layer 2 — Statistical validation (per bar)

Price deviation: $|p_t - p_{t-1}| / \sigma_{20d} > \theta \rightarrow$ flag. Volume spike: $V_t > 5 \times \text{ADV}_{20d} \rightarrow$ flag. Zero-volume bar during exchange hours $\rightarrow$ flag.

5.0.3 Layer 3 — Sequence validation (per session)

Expected bar count vs. actual bar count; gap detection with configurable tolerance; stale detection for N+ consecutive identical prices.

5.0.4 Layer 4 — Cross-source validation (daily)

Compare the same instrument across two or more independent vendors. Price discrepancy $>$ 0.1% on adjusted close $\rightarrow$ investigate. Corporate action date discrepancy $\rightarrow$ manual review queue.

Here is a minimal Python implementation:

```
import pandas as pd
import numpy as np
from dataclasses import dataclass, field
from typing import Optional

@dataclass
class DataQualityChecker:
    series: pd.Series
    expected_freq: str = "1min"
    spike_threshold: float = 3.0
    stale_n: int = 5
    _issues: dict = field(default_factory=dict)

    def check_spike(self) -> pd.Index:
        returns = self.series.pct_change().dropna()
        roll_std = returns.rolling(20).std()
        spikes = returns.index[np.abs(returns) > self.spike_threshold * roll_std]
        self._issues["spikes"] = len(spikes)
        return spikes

    def check_gaps(self) -> pd.DataFrame:
        full_idx = pd.date_range(
            self.series.index[0], self.series.index[-1], freq=self.expected_freq
        )
        missing = full_idx.difference(self.series.index)
        gaps = pd.DataFrame({"timestamp": missing, "size": 1})
        self._issues["gaps"] = len(missing)
        return gaps

    def check_stale(self) -> pd.Index:
        diffs = self.series.diff().abs()
        stale_mask = diffs.rolling(self.stale_n).sum() == 0
        self._issues["stale_bars"] = int(stale_mask.sum())
```

```
        return self.series.index[stale_mask]

    def summary_report(self) -> pd.DataFrame:
        _ = self.check_spike()
        _ = self.check_gaps()
        _ = self.check_stale()
        total = len(self.series)
        rows = [
            {"issue": k, "count": v,
             "pct_of_series": round(v / total * 100, 3),
             "severity": "HIGH" if v / total > 0.01
                         else "MEDIUM" if v / total > 0.001 else "LOW"}
            for k, v in self._issues.items()
        ]
        return pd.DataFrame(rows)
```

6 The vendor problem nobody talks about.

No data vendor has zero data quality issues. The distinction that matters is not Tier 1 vs. free — it is **known failure modes vs. unknown failure modes**.

Tier 1 vendors (Bloomberg, Refinitiv) have dedicated data operations and systematic corporate action processing. They still have issues — particularly in non-US markets, small caps, and OTC instruments. The gaps are documented and often correctable. Free or low-cost vendors have systematic gaps that are acceptable for research but categorically not for production capital allocation.

The most dangerous segment: **paid vendors with institutional branding but retail-grade data operations**. The feed looks credible until a corporate action is wrong, and that error persists in your backtest for five years of history.

The only defense is cross-vendor validation. For any strategy approaching production, validate your primary data source against at least one independent vendor for the full backtest period before committing capital. Price discrepancies above 0.1% on adjusted close warrant investigation; discrepancies on corporate action dates require manual reconciliation.

7 Key takeaways.

1. Dirty data systematically inflates backtest Sharpe by **0.2–0.5** in typical equity momentum strategies — this is not theoretical, it is measurable.
2. The gap between backtest and live performance is more often a **data quality problem** than an alpha decay problem.
3. Forward fill is the default but not the best: LOCF introduces artificial autocorrelation that inflates trend-following signals; **Kalman filter** handles gaps without this artifact.
4. **Corporate action errors** have the largest single-step impact on long-term backtest accuracy — a 0.24 Sharpe degradation in the example above.
5. **Survivorship bias** alone can inflate Sharpe by 15–30% depending on strategy period and universe scope.

6. Production data validation requires **four independent layers**: schema, statistical, sequence, and cross-source — any single layer alone is insufficient.

8 Constraints and limitations.

- Sharpe degradation figures are illustrative and calibrated to academic literature ranges — actual impact is highly strategy-dependent (HFT vs. daily vs. weekly rebalancing).
- Kalman filter implementation requires calibration of process and observation noise — wrong parameter assumptions introduce their own form of bias.
- Cross-vendor validation is expensive and may not be feasible for exotic instruments or niche data sources.
- Survivorship bias correction requires a point-in-time historical universe — this is not available from all vendors.
- ML-based imputation introduces model risk directly into the data layer and should be used with caution in production environments.

How does your team currently measure data quality before running a strategy in production? We are building a public benchmark of data quality metrics across asset classes — if you have data to contribute, we would like to compare notes.

If you are running production strategies and want to benchmark your data quality pipeline against what we have built — or if you have hit specific data issues that cost you in production — we are *genuinely interested in comparing notes*.

Related reading

- 06 — Quantum supremacy in finance: marketing or reality?
- 05 — No, you didn't build the next Bloomberg in a weekend
- 04 — Running 36 AI agents in production
- 03 — CeDeFi is not a buzzword
- 02 — Why finance still doesn't have a proper data layer
- 01 — AI Agents in Trading: where the real boundary lies